empeo Registration Automation

How to use and conclusion · Gofive Automation Tester Exam 1.0

System under test	https://portal.uat.gofive.co.th/Register/empeo (UAT)
Stack	Playwright Test 1.56.1 · TypeScript 5.6 · Node.js 22 · Chromium
Pattern	Page Object Model · custom fixtures · data-testid locators · text-as-constants
Deliverables	01_Demo · 02_Testcase · 03_Automate_Code · How_to_use_and_Conclusion.pdf

42	42 / 42	~1.3 min	4
automated test runs	passed, no retries	full suite, 4 workers	defects found

1. Submission package

Item	Content
`01_Demo/`	`REG-E2E-01-happy-path.mp4` : recording of the automated happy path (form → promo → OTP → password → success)
`02_Testcase/`	`TEST_CASES.xlsx` (Summary · Test Cases · Defects) and `TEST_CASES.md`
`03_Automate_Code/`	Playwright + TypeScript project, ready to run locally or in CI
`How_to_use_and_Conclusion.pdf`	This document

2. Prerequisites

Item	Requirement
Node.js	v20 or newer (tested on v22). Check with `node -v`
npm	Comes with Node.js
OS	macOS, Windows or Linux
Network	Access to `portal.uat.gofive.co.th` and `portal-uat-api.gofive.co.th`

3. Installation

1. Unzip `gofive-empeo-registration-automation.zip` and open a terminal in the code folder.

```
cd gofive/03_Automate_Code
```

2. Install the dependencies. Versions are pinned in `package-lock.json` .

```
npm ci
```

3. Download the Chromium browser used by Playwright.

```
npx playwright install chromium
```

4. Optional: confirm the project compiles.

```
npm run typecheck
```

4. Running the tests

Command	What it does
`npm test`	Full suite: form validation, promo code, OTP and end-to-end registration
`npm run test:ui-only`	Everything except live submissions (`@e2e` excluded). Creates **no** data on UAT
`npm run test:e2e`	Only the live registration, OTP and create-password tests
`npm run test:smoke`	Critical path only (`@smoke`): empty submit, valid promo, full happy path
`npm run test:headed`	Full suite in a visible browser, one test at a time
`npm run test:demo`	Smoke tests in a visible browser, slowed down, with video recorded

Run a single test or group

```
npx playwright test -g "REG-E2E-01"          # by test case ID
npx playwright test tests/promo-code.spec.ts # by file
npx playwright test --ui                     # interactive UI mode
```

Note: each full run (`npm test`) creates 7 new trial registrations on UAT. Company name and email domain are unique per run, so the suite can be re-run any number of times without data clean-up. Use `npm run test:ui-only` when you do not want to create any UAT data.

5. Viewing results

HTML report

```
npm run report
```

This opens `playwright-report/index.html` . Every test case ID is shown with its status, duration and steps. A failing test also includes its screenshot, video and trace.

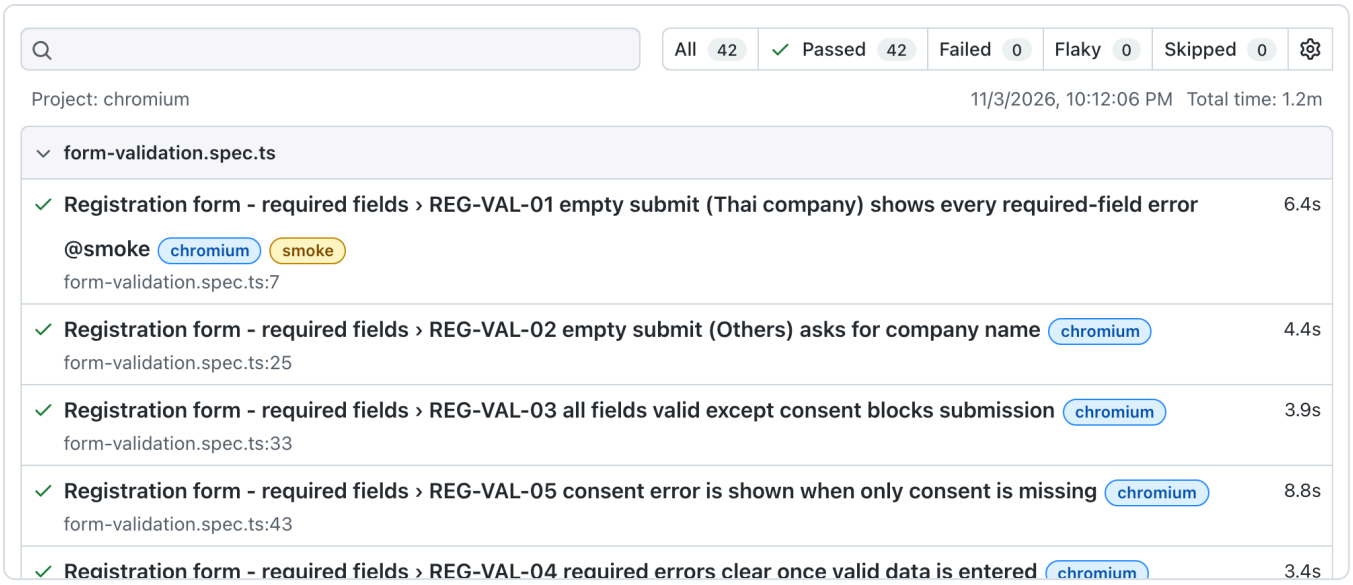


Figure 1 – Playwright HTML report: 42 passed, 0 failed, 0 flaky.

Debugging a failure

```
npx playwright show-trace test-results/<test-folder>/trace.zip
```

The trace viewer replays every action with DOM snapshots, network calls and console logs. A JUnit file for CI tools is written to `test-results/junit.xml`.

Demo video

`01_Demo/REG-E2E-01-happy-path.mp4` records the full happy path: form → promo `FREE15DAY` → OTP `123456` → password rules → success page.

To record a new video:

```
npm run test:demo
```

The videos are saved as `test-results/**/video.webm`.

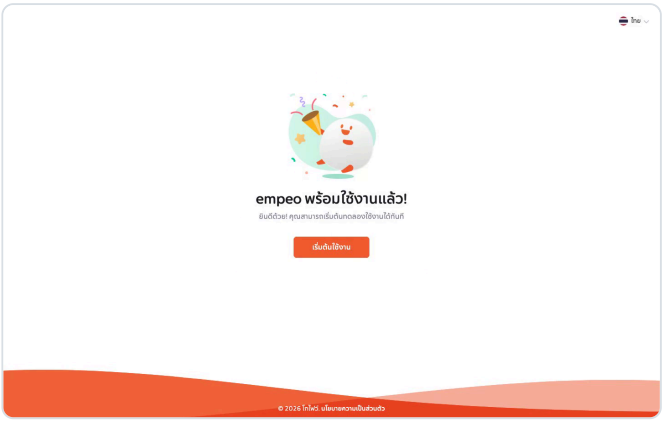


Figure 2 – Last frame of the happy-path video.

6. Configuration

Every value has a default, so nothing needs to be configured to run. To override one, set an environment variable, for example `TEST_PROMO=OTHERCODE npm test`.

Variable	Default	Purpose
`BASE_URL`	https://portal.uat.gofive.co.th	Web app under test
`API_BASE_URL`	https://portal-uat-api.gofive.co.th	Backend host used for request guards
`TEST_PHONE`	0967690708	Fixed mobile number

`TEST_OTP`	123456	Fixed OTP
`TEST_PROMO`	FREE15DAY	Valid promo code
`TEST_PASSWORD`	EmpeoQa2026	Password set on the create-password page
`REGISTERED_EMAIL`	admin@gofive.co.th	Existing account for the duplicate-email test
`REGISTERED_DOMAIN`	mailinator.com	Existing domain for the duplicate-domain test
`RECORD_VIDEO`	unset	`1` = record video for every test

7. Project structure

```
03_Automate_Code/
├── src/
│   ├── config/env.ts              environment and fixed test data
│   ├── data/messages.ts           expected UI messages (text as constants)
│   ├── data/registrant.ts         unique registrant factory, phone helpers
│   ├── fixtures/test.ts           custom fixtures, third-party traffic blocking
│   └── pages/
│       ├── RegisterPage.ts        registration form
│       ├── OtpDialog.ts           OTP dialog
│       └── CreatePasswordPage.ts  create password and success page
├── tests/
│   ├── form-validation.spec.ts    required fields, email, phone, limits, company type
│   ├── promo-code.spec.ts         valid / invalid / case / remove / reused
│   └── registration-e2e.spec.ts   happy path and OTP scenarios (@e2e)
├── .github/workflows/registration-tests.yml
└── playwright.config.ts
```

8. Test coverage summary

Area	Tests	Scenarios
Required fields	5	Empty submit (Thai / Others), consent missing, errors clear after correction
Company type	2	Thai ↔ Others toggle, tax-ID search and selection
Email	8	Malformed ×4, valid, already registered, domain already registered, public provider
Mobile number	6	Too short, too long, wrong prefix, letters, valid with and without leading 0
Length limits	5	Email 150, first / last name 255, phone 15, company 255
Promo code	9	Valid, case-insensitive, unknown ×2, special characters, remove, max length 50, reused (stub), isValid=false (stub)
OTP	6	Dialog content, incomplete, non-numeric, wrong OTP, resend, expired (stub)

End-to-end	1	Register → OTP → password rules → success page

The full case list with steps and expected results is in `02_Testcase/TEST_CASES.xlsx`.

9. Reliability design

- **Unique data per run:** the backend rejects reused emails and reused company email domains, so every registrant gets a fresh company name and domain.
- **Stable locators:** the app's `data-testid` attributes are used everywhere. No index or CSS-path locators.
- **No fixed waits:** the suite waits on real signals: API responses, the form leaving `ng-pending`, URL changes, and web-first assertions.
- **Guarded negative submits:** tests that must not register abort every non-validation backend call, so a client-side bug cannot silently create data.
- **Third-party traffic blocked:** analytics, ads and heatmap scripts are aborted, which makes runs faster and steadier.
- **Stubs only where needed:** expired OTP and exhausted promo code cannot be reproduced on demand. All other tests use the live UAT API.
- **Known defects tracked:** confirmed defects are marked `test.fail`, so CI stays green. When a fix ships, the test turns red to signal that the marker should be removed.
- **CI-ready:** GitHub Actions runs on push and PR and daily at 06:00 (Asia/Bangkok), with 2 retries, HTML and JUnit reports, and artifacts.

10. Defects found

#	Severity	Finding
D-01	Medium	With only the consent checkbox missing, submit does nothing: no message, no request.
D-02	Medium	The coupon UI shows the code as applied even when the API returns `isValid: false`.
D-03	Low	OTP business errors (e.g. expired) show the raw text "Business error with 1 errors."
D-04	Low	The create-password button is enabled when password and confirmation do not match.

11. Troubleshooting

Symptom	Fix
`Executable doesn't exist`	Run `npx playwright install chromium`
Timeouts on OTP verification	UAT verification can take up to ~40 s. The suite allows 90 s. Re-run, or check UAT availability.
Duplicate email / domain errors	Make sure `REGISTERED_EMAIL` / `REGISTERED_DOMAIN` still exist on UAT, or override them.
Tests fail behind a proxy or VPN	Allow both UAT hosts listed in section 1.

12. Conclusion

42	40	2	0
test cases automated	pass as expected	fail = confirmed defects	flaky across repeated runs

Result

The empeo registration flow works end to end on UAT. A new company can register with the promo code `FREE15DAY`, verify OTP `123456`, set a password and reach "empeo พร้อมใช้งานแล้ว!". Field validation covers required fields, email format, duplicate email/domain, Thai mobile format and field length limits, and it behaves correctly.

Coverage against the exam scope

Requested scope	Covered	Test cases
Successful registration with valid data	Yes	REG-E2E-01
OTP verification flow	Yes	REG-OTP-01 to 05, REG-E2E-01
Promo code validation	Yes	REG-PRM-01 to 08
Required fields and error handling	Yes	REG-VAL-01 to 05, REG-CMP-01/02
Invalid phone numbers	Yes	REG-VAL-20/21
Expired OTP	Yes (stub)	REG-OTP-06
Reused promo code	Yes (stub)	REG-PRM-06/07

Key findings

- Four defects found. Two are Medium: the consent checkbox gives no feedback when it is the only thing missing, and the coupon UI trusts the HTTP status over `isValid`. Two are Low: a raw backend error appears on OTP failure, and password/confirmation mismatch is not blocked.
- The backend blocks duplicate company **email domains**, not only duplicate emails. Test data must therefore use a fresh domain on every run.
- Correct-OTP verification on UAT sometimes takes ~40 s. This may be worth monitoring as a performance item.

Limitations and next steps

- The Thai-company end-to-end submit is not automated, because each real tax ID can be registered only once. Recommendation: provide a resettable test tax ID or a data clean-up API on UAT.
- Expired OTP and exhausted promo code are covered by API stubs. Recommendation: provide a short-TTL OTP setting or a dedicated exhausted coupon on UAT, so these can run against the real backend.
- Possible extensions: run cross-browser (Firefox / WebKit / mobile viewport), cover the English locale, and add API-level contract tests for `/api/v2/register`, `verify-otp` and `coupon-validation`.